

Master Migration Summary Unified: LAN a SAP (Estado Global y Arquitectura)

Documento Maestro Unificado

Proyecto: Migración LAN (Intelisis) a SAP S/4HANA (Módulo Órdenes, E-Commerce y Crédito)
Stack Tecnológico: C# .NET 4.7.2 (Web API), SAP OData V2, SQLite, SQL Server (SigMavi / Android)
Objetivo: Desacoplar la dependencia del ERP heredado (Tablas locales y SPs de Intelisis) hacia la nueva arquitectura orientada a microservicios OData de S/4HANA, y permitir a DMZ Magento conectarse al nuevo **ServicioSAP**.
Última Actualización: 13 de Julio de 2026

Este documento unifica el análisis de brechas (Gap Analysis) y el plan de implementación de todos los módulos que componen el ecosistema de la migración. (Single Source of Truth) para entender el avance global del proyecto.

1. Reglas Arquitectónicas Core (Reglas de Oro)

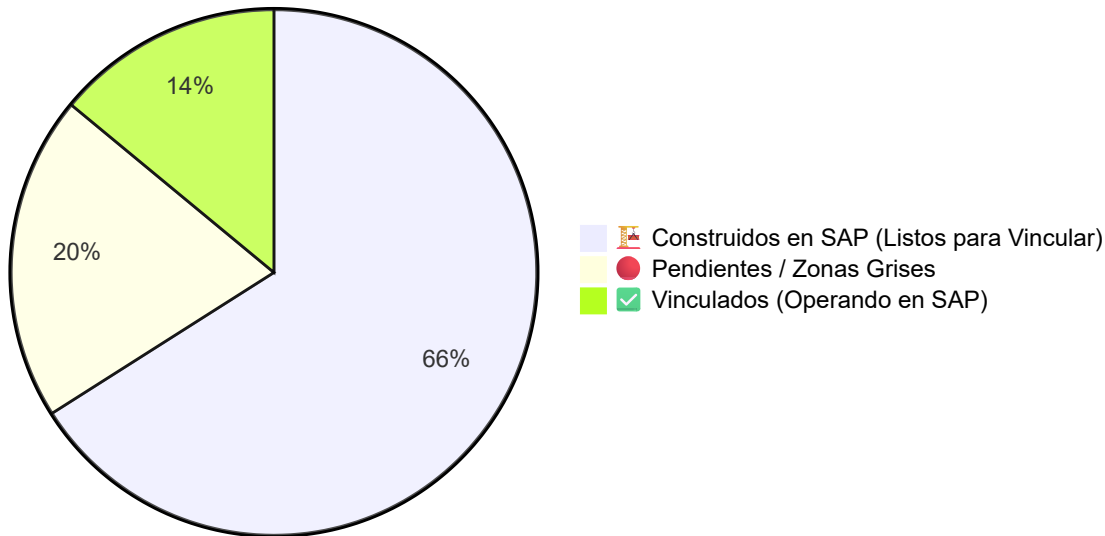
- Cero Consultas SAP DB (OData First):** Supresión total de sentencias `INSERT/SELECT` contra tablas Z. Todo opera bajo protocolos OData (EntitySets) con inyección dinámica de credenciales OAuth 2.0 y base URL.
- Supresión del Motor Legacy (`spAfectar`):** El "corazón" de Intelisis ha muerto. S/4HANA asume nativamente la deducción de inventario (ATP), Cuentas por Cobrar y Contabilidad mediante los módulos SD, FI y MM.
- Bases Locales Permanentes (Zonas Grises de SAP):**
 - SQLite:** Mantiene los rastros transaccionales de OpenPay y Servicio de Guías, los cuales no viajarán en crudo a SAP.
 - MAVICBOSANDROID (SQL Server):** Mantiene la validación rigurosa de SMS y Alta de Solicitudes de Crédito Web.
- Refactorización Asíncrona (Async/Await):** Para evitar *Socket Exhaustion* bajo carga (ej. Hot Sale), todo el monolito HTTP a SAP se migra progresivamente a `async Task`. (Actualmente completadas Fases 1 a 3; pendientes `Orders`, `Credit`).

Avance de Integración: DMZ vs SAP (Visión Real de Desarrollo)

Para mantener una visualización justa y clara del esfuerzo de migración, este gráfico muestra el estado real del backend (`ServicioSAP`) respecto a las dependencias heredadas de Intelisis en la DMZ:

-  **Completados y Vinculados (DMZ → SAP): 7 Endpoints (14%)** operando sobre `curl.PostSAP(...)`.
-  **Construidos en SAP (Esperando Switch): 33 Endpoints (66%)**. APIs de Abonos (EX01/TZ01), SMS, OpenPay, Cupones, Direcciones y Monedero que ya fueron desarrolladas en ServicioSAP y sólo esperan que DMZ apunte hacia ellas.
-  **Pendientes / Bloqueados (Zonas Grises): 10 Endpoints (20%)**. Flujos que aún no se han desarrollado por falta de definición (ej. Store Pickup, Mutación SKUs Celulares, CRM Quejas). (Nota: *Excluye CrediLana*).

El Desarrollo: DMZ vs ServicioSAP (Total: 50 Endpoints)



2. Análisis de Brecha Crítica: DMZ ↔ SAP (Test Release Bloqueos)

Para que Magento y DMZ puedan realizar pruebas E2E apuntando 100% al nuevo ServicioSAP y desechar el ruteo a LAN, se enfrentan los siguientes Gaps:

● Módulos Ya Migrados y Soportados

- **Creación de Órdenes (`order/new`):** El payload hacia `ZAPI_SALESORDER_SRV` está consolidado (JSON atómico `OrderHeaderIn`, `to_items`), con inyección obligatoria de `to_return: []`. El `hardcode (SalesOrg: 04/05, Plant: 0504/0505/2000)` en `BuildSapOrder` está identificado para su dinamización.
- **Creación y Actualización de Business Partners:** El bug de Calendario de Fábrica (`TFACD`) fue superado inyectando `"Perrl"`: `"AM"`. El BP se crea exitosamente.
- **Direcciones de Envío:** Las 6 APIs de Business Partner Address (Python) han sido traducidas y acopladas en `PartnerAddressController` y vinculadas nativamente al flujo de `SetOrder`. **Hitos técnicos logrados:**
 - Corrección del OData `$select` para obligar a SAP a retornar el nodo `to_PhoneNumber` expandido.
 - Corrección del Deep Insert en `OrderMethods.cs`: Se inyectó `ToPhoneNumber` durante la creación del BPAddress para evitar errores 404 en futuras actualizaciones (`UpdatePhone`).
 - Validación funcional y exitosa de `ChangeSalesDocumentAddressSet` (Vinculación de `AddressID` a `SalesDocument`), confirmando que C# maneja correctamente tanto el Happy Path como las excepciones de negocio nativas de SAP (ej. *"Destinatario no actualizado en área de ventas"*).
- **Validación de Identidad (Reemplazo `StartsWith("C")`):** Identificación cruzando los campos `cuenta` y `cliente`.

● Módulos Faltantes para el Release Test (Gaps de DMZ)

1. **El Esqueleto de S/4HANA Core (SD):** Faltan por reconstruir en C# consumiendo SAP los endpoints de estatus de pedido (`getIntelisisStatuses`), cancelación de órdenes (`cancelOrder`) y Devoluciones/RMA.
2. **El Bloque Crítico de Crédito (`CreditController`):** DMZ tiene 25 endpoints apuntando a LAN. **Aclaración Importante:** Todo lo relacionado exclusivamente a flujos de "CrediLana" (solicitudes web y su frontend) **NO** forma parte de la migración a SAP. El alcance se restringe **únicamente** a reemplazar la dependencia que existía con Intelisis. **Falta que ServicioSAP:**
 - Conecte a SAP FI-CA para reemplazar las consultas de saldos a Intelisis y exponer `getClientesSaldo` y `getClientesFactura`.
 - Conecte a la DB local (`MAVICBOSANDROID`) para procesar `validateSms` y las solicitudes (excluyendo lo propio de CrediLana).
3. **Persistencia Local de Pasarelas:** DMZ requiere las rutas `ManagePaynetOrders` e `insertPaymentData`, las cuales exigen que SQLite esté implementado en ServicioSAP.

3. Inventario Técnico de Integraciones (SAP OData)

Módulo SD (Ventas y Logística)

- ✓ **SD01 (Pedidos):** Activo. Reemplaza `VentaD` y `SP_eCommerceNuevoPed`.
- ✓ **SD29 (Properlist):** Activo. Calcula condiciones de precio para crédito.
- ✓ **SD36 (Consulta de Documentos):** Activo. Protege contra pedidos duplicados (`Idempotencia ZSD_{docType}_{folio}`).
- ✓ **TZ01 (Splits ZMER/ZPRE/ZSEG):** Activo y conectado a DMZ. Procesa parcialidades y abonos.
- ✓ **SD09/SD46/SD48 (Reversiones):** Completado. Programación de devoluciones y anulación de facturas (Realizado por Javier Alvarado).

Módulo BP y Finanzas (FI/CA)

- ✓ **BP01, BP02, BP05:** Activos. Reemplazan a `SpCREDIDatosSolicitudCreditoArt`.
- ✓ **SD33 / SD18:** Activos. Extraen el saldo real de bonos y monedero electrónico.
- ✓ **EX01 (Documentos No Compensados):** Activo y conectado a DMZ. Permite al cliente observar deudas detalladas (partidas abiertas FI).
- ⚠ **Pagos en Línea y Cuenta Clabe STP:** Pendiente de consulta con Álvaro para determinar la cuenta y canales a mostrar en interfaz.

Módulo DM (Maestro de Materiales y Logística ATP)

- ✓ **DM01 / DM02 / DM04 / DM05:** Integrados. Extraen maestros de catálogo, jerarquías y etiquetas SEO.
- ✓ **DIM11 (ATP / Inventario Real-time):** Reemplaza a `eCommerceExist`. Consulta existencias en S/4HANA antes de permitir la orden.
- ✓ **DM07 (Sucursales):** Activo. Extrae el ID de la tienda para vincularlo al promotor/cupón, erradicando el JOIN heredado contra la tabla local `Sucursal`.

4. Zonas Grises y Faltantes Funcionales (Bloqueadores de Negocio)

Estas tres áreas requieren una respuesta inminente de los arquitectos de negocio y de los consultores SAP para proceder a programarlas:

- Mutación Regional de SKUs (Celulares):** En Intelisis se agregaba `-R5` a teléfonos según la tabla `VTASCCodigoPostalRegionCelular`. ¿Lo asumirá SAP con el módulo de Sustitución de Materiales, o C# debe clonar la regla leyendo las nuevas tablas maestras?
- Pickup en Tienda (Recolección):** Legacy usaba `TrWDM0285_CteRecoge`. ¿Estos datos de quién recolecta se enviarán dentro del JSON de SD01 (ej. Observaciones `TEXT_ID = Z001`) o se mantendrán en la BD SigMavi?
- Tablas Legacy Pendientes de Mapear:** Existen más de 20 tablas de Intelisis (ej. `VTASCPProductoControl`, `eCommerceLineasRelacion`) que no tienen claro su destino en S/4HANA.
- Cupones Promocionales (AWS Catalog):** La lógica C# ya está finalizada, pero AWS reporta códigos legacy (ej. `D40`) mientras la nueva API devuelve `20000001`. El catálogo debe homologarse antes de lanzar a productivo.

5. Roadmap Sugerido de Ejecución

Para un cierre exitoso del Release Test, el orden de ataque técnico es:

- Esqueletos Híbridos para DMZ:** Clonar todas las firmas de endpoints que falten de DMZ hacia ServicioSAP. Redirigir la base de datos `MAVICBOSANDROID` temporalmente a ServicioSAP para procesar validaciones SMS y mitigar bloqueos del flujo de crédito.
- Dinamización de Valores:** Quitar el *hardcode* (`01`, `0090`) de `OrdersController` para que S4 reciba la data del frontend de Magento (ej. `04`, `0504`).
- Configuración Entity Framework (SQLite):** Configurar `openpay_orders` y `servicio_guias` en la infraestructura del nuevo backend.
- Refactorización Core (Async Fase 4 y 5):** Transformar la bestia monolítica de `OrderMethods.cs` a `async Task` para destrabar el cuello de botella HTTP.

ANEXO HISTÓRICO Y DETALLES DE MIGRACIÓN

****# Master Migration Log: Órdenes de LAN a SAP

Este documento consolida el historial, decisiones arquitectónicas, planes de implementación y walkthroughs de todas las sesiones (del 02 al 12 de Junio). Mantiene un único punto de verdad centralizado del proyecto `ServicioSAP` .

1. Reglas Arquitectónicas Core (Aprobadas)

- **Cero Consultas Directas a SAP DB:** C# no hará sentencias `INSERT/SELECT` contra tablas `Z` de SAP. Toda persistencia usa `OData` (`SD`, `BP`, `MM`) a través de DTOs mapeados (`OrderModel` , etc.) y autenticación `OAuth 2.0` (`CSRF`).
 - **Intelisis Deprecado:** Las tablas `Venta` , `VentaD` , `Cte` , `CteTel` , y el uso del `ADO.NET` contra ellas, quedan deprecadas.
 - **SQLite (OpenPay / Guías):** Se mantiene **temporalmente** para guardar rastros de webhooks y guías. *(Se descarta el plan del 03/06 de eliminarlo inmediatamente).*
 - **Android DB (MAVICBOSANDROID):** Se mantiene la conexión directa para la validación de SMS (`TcAAEA00030_EnvioMensajes`) y la creación de la solicitud de crédito web (`VTASdArtCreditoWeb`).
-

2. Progreso de Migración por Módulos (Estado Actual)

2.1 Flujo Base y Contado (Casi Completo)

- **Idempotencia (Sustitución de `obtenerIdVenta`):** Completado (12/06). Se implementó la API **SD36** al inicio de `SetOrder` estructurando el `PurchNoC` (`ZSD_{docType}_{folio}`) para detectar y rechazar pedidos duplicados originados por Magento.
- **Mapeo Payload SD01/SD04:** `BuildSapOrder` fue adaptado para enviar un JSON atómico (`OrderHeaderIn` , `to_items` , etc.), eliminando la transaccionalidad `VentaD` .
- **OpenPay y Guías:** `SaveToValidateOpenpay` , `SaveOpenpayStoresOrder` y `SaveGuide` ya están migrados atacando `SQLite local`.

2.2 Módulo de Crédito y SMS (Completado)

- **Validación SMS (`IsValidated`):** Implementada como validación cruzada. Consulta SAP **BP05** (`BusinessPartnerMethods.GetClient`) para el cliente, y la base `MAVIANDROID01` para confirmar el token del teléfono.
- **Creación de Solicitud Android:** `CrearSolicitudCredito` mapea más de 70 variables (incluyendo lada/teléfono validado SMS).
- **Callback a DMZ Magento (16/06):** Si el cliente es nuevo (se valida evaluando si SAP no devolvió cuenta BP pero sí existe un ID de Magento), se detona un hilo asíncrono para liberar el crédito y notificar el callback de autorización hacia Magento DMZ usando las credenciales legadas (`USER_DMZ` / `URL_DMZ`), conservando el Auth JWT en dos pasos, sin invocar `OAuth` de SAP `S/4HANA`.
- **Cálculo e Inserción de Artículos:** Refactorizado el 11/06. Se eliminó el uso de `SpVTASInsertArtSolCreditoLinea` . Ahora C# consume la **SD29 (Properlist)** para extraer precios e inserta de forma nativa en `VTASdArtCreditoWeb` (`SQLite/Android`). Se respetó la regla quemada para el Flete (`SEGU00001`).

2.3 Módulo de Saldos y Monedero (Completado)

- **Consulta de Saldo (`CheckClientBalance`):** Se eliminó el `SpCREDIDatosSolicitudCreditoArt` . Ahora se extrae de la propiedad `Zcrimporte` (`Exposición / Saldo`) y `ZlimCred` devolviendo el JSON del **BP05**.
- **Monederos:** Se eliminó la dependencia local de `xpVerificarMovMonederoMAVI` . El cobro se procesa mediante el campo `Zredimepuntos` del Payload de la Orden. Las vistas previas consumen **SD33** (`AccountMethods.GetBonus`).

2.4 Módulo de Direcciones de Envío (Completado - 29/06)

- **Migración de Python a C#:** Se migraron las 6 APIs (`A_GET`, `A_POST`, `A_PATCH`, `UpdatePhone`, `AS_GET`, `AS_POST_SALESDOC`) desde los microservicios en Python (`salesanddistribution-dev` y `businesspartner-dev`) hacia el proyecto monolítico `.NET 4.7.2` (`ServicioSAP`).

- **Validación de Paridad:** Se analizó a profundidad el código y se confirmó que el alcance exclusivo para "Direcciones de Envío" se encuentra **100% adaptado** en C#, cubriendo las 6 rutas obligatorias de forma idéntica a Python (incluyendo mapeo de fechas SAP `/Date(epoch)/`).
- **Arquitectura:** Se aislaron en un nuevo `PartnerAddressController` y `DeliveryAddressMethods`.
- **Flujo de Órdenes (SetOrder):** Se refactorizó `DatosEntregaInsert` en `OrderMethods.cs`. Ahora consume nativamente las APIs OData para insertar/vincular el domicilio al pedido, eliminando para siempre las escrituras en la tabla local legacy `DM0312DatosEntrega`.
- **Smart Matching:** Se implementó una lógica inteligente que consulta (`A_GET_BusinessPartnerAddress`) las direcciones previas del Business Partner antes de crear una nueva, reutilizando el `AddressID` histórico si coincide el CP y la Calle, evitando ensuciar la base de datos de SAP.
- **Compatibilidad de Serialización Web API:** Se unificó el uso de `Newtonsoft.Json` (reemplazando `System.Text.Json`) en `DeliveryAddressMethods` y `OrderMethods` para garantizar la compatibilidad nativa con el motor de respuesta de ASP.NET 4.7.2, resolviendo el bug que arrojaba `{"ValueKind": 2}` al cliente y logrando un casteo dinámico perfecto del `AddressID`.
- **Testing Parcial (En Progreso - 06/07):** La ruta de consulta de direcciones (GET) y la ruta de **creación de direcciones (POST)** fueron validadas satisfactoriamente en entorno local devolviendo el `AddressID` nativo de SAP. Resta validar las peticiones de actualización (PATCH) y la vinculación a la orden (AS_POST_SALESDOC) en el flujo final de `SetOrder`.

2.5 Módulo de Cupones Promocionales (Completado - 01/07)

- **Desacoplamiento de Intelisis:** Se migró la lógica del SP `SpVTASeCommerceValidaPromotor` y `SpVTASeCommerceActCupon` a métodos C# puros (`HandlePromoCode`).
- **Integración API Android:** La validación de empleados/promotores se hace ahora consumiendo el endpoint `/employees/get_personalById` de la API de Android.
- **Integración API AWS Catalogs:** La validación de departamentos y puestos permitidos ("Código de promotor") ya no usa la vieja `TablaStd`, sino el endpoint `GetConfiguracionCatalogo` de AWS.
- **Nuevo Endpoint (ValidateCupon):** Se expuso la ruta HTTP GET `/order/validatecupon/{codigo}` en `OrderController` para simular la verificación en vivo que requiere Magento, con respuesta `OK` o error HTTP 400.
- **Gap de Datos Detectado:** Se documentó un desfase donde el catálogo actual (AWS) mantiene códigos heredados de Intelisis (ej. `D40`), mientras que la nueva API de Android devuelve claves de SAP SuccessFactors (ej. `ALMACEN (200000001)`). Se espera definición del negocio para actualizar AWS o relajar la regla a `.Contains()`.

3. Puntos Críticos y Bloqueos Pendientes (Análisis del Líder Técnico)

Las siguientes dependencias legadas detienen la finalización al 100% del motor de órdenes:

✓ A. El Conflicto de `spAfectar` (Resuelto y Eliminado)

- **Decisión Tomada (16/06):** El SP `spAfectar` iba a Intelisis y queda DESECHADO Y ELIMINADO DEFINITIVAMENTE. Toda la afectación (inventario, cuentas por cobrar, contabilidad) es asumida de forma nativa por SAP S/4HANA.
- **Protocolo SigMavi:** Para otros SPs/Tablas, la conexión ya no sería a Intelisis sino a `SigMavi` usando la DLL. Por tanto, antes de borrar lógicas como la de Cupones, se debe **consultar con el usuario** si la tabla fue migrada a `SigMavi` o si se reemplazará por una petición SAP.

⚠ B. Datos de Entrega y Pickup (`SpWDM0285_CteRecoge` / `DM0312DatosEntrega`)

- **Datos de Entrega (Resuelto 26/06):** La parte de envío a domicilio ya fue migrada. Toda la persistencia local de Intelisis fue desechada. Ahora C# consume de manera nativa los endpoints OData de SAP (creando el BP Address y vinculando el `AddressID` al `SalesDoc`).
- **Pickup en Sucursal (Pendiente):** Falta validar por parte del equipo si el flujo "Recoger en Tienda" también será absorbido nativamente por OData o si requerirá persistencia en `SigMavi`. No se programará nada de Pickup hasta recibir la definición.

⚠ C. Validaciones Especiales de Partidas (`SpVTASeCommerceDetPedidos`)

- **Análisis Profundo (11/06):** Se examinó el interior del SP. Su función principal era insertar partidas temporalmente en la tabla `eCommerceDetPedidos` (ya deprecada por SAP) y validar que el precio enviado no fuera menor al del catálogo.
- **Regla Oculta Encontrada:** Si el artículo es de la familia '**TELEFONIA**', el SP interceptaba el SKU y lo mutaba por un SKU Regional (`Region5` o `Region6`) cruzando el Código Postal (`VTASCCodigoPostalRegionCelular`), `VTASCCRegionSku`, y validando

existencias en `eCommerceExist`.

- **Decisión Esperada:** ¿La mutación regional de los SKUs de celulares la va a manejar SAP de forma nativa con "Sustitución de Materiales" (MM), o necesitamos migrar a C# toda la lógica de cambio de SKU por Código Postal antes de armar el Payload de la Orden?

✓ D. Cupones Promocionales (`SpVTASVentaCupon`) (Resuelto y Migrado - 01/07)

- **Decisión Tomada y Aplicada:** Se acordó mantener la lógica en el middleware (C#) y no depender de SAP Pricing para esto, dado que se requiere inyectar registros para el pago de comisiones de tiendas de internet.
- Toda la dependencia hacia `Personal` y `TablaStd` fue reescrita para consumir microservicios modernos (Android / AWS).
- Se habilitó la lógica `QuemarCupon` dentro de `SetOrder` para afectar la tabla local de `VentasCupones` y solicitar el `Werks` vía la API OData del Business Partner (`AS_GET_ZQBP_AGENTE`), asegurando un registro transparente para Magento.
- **Pendiente Operativo:** El equipo de operaciones debe actualizar las reglas del catálogo de "Código de promotor" en AWS para igualarlas a los valores de la nueva API de Android (ej. cambiar `D40` por `20000001`).

4. Sigüientes Pasos de Implementación (Para la próxima sesión)

1. Ensamblar el Payload Final (C#):

- Terminar de codificar el método que construirá la clase `SD01SalesOrderRequest` en `BuildSapOrder()`.
- Inyectar ahí los artículos cotizados, el cliente validado por BP05 y el flag de descuento de monedero (`Zredimepuntos`).
- Finalizar la prueba de envío e integración del JSON completo hacia SAP.

3. Programar las Zonas Grises (Depende del Líder):

- Codificar la solución a las dudas de la Sección 3 (Celulares, Pickup, Cupones y Afectación) tan pronto el líder indique el camino a seguir.

4. Nuevos Contextos Agregados (12/06):

- Se añadió la documentación de Payloads (`magento_payloads.md`) para las estructuras de Banktransfer (1ra compra nula), Openpay Stores y Openpay Cards, como guía técnica constante en el skill.

5. Análisis de Dependencias SPs (Añadido):

- Se generó la matriz exhaustiva de Stored Procedures (`lan_sp_dependencies.md`) la cual mapea cada SP principal del flujo eCommerce con sus tablas afectadas y los sub-SPs invocados (ej. dependencias internas hacia `spAfectar` o `spRedimirMovMonederoMAVI`). Esta matriz servirá como documento base para la validación de qué componentes heredados deben migrarse a `SigMavi` y cuáles desaparecen por S/4HANA.

6. Zonas Grises (Actualización 02/07):

- La **Mutación de SKUs (Telefonía)** y el **Pickup en Sucursal** se documentaron en el plan maestro como pendientes de implementación.
- **ESTADO:** *Bloqueado / Pendiente.*
 - **SKUs y Telefonía:** Está pendiente porque aún no se ha proporcionado el nombre de las tablas o APIs que reemplazarán a las de legacy (`VTASCCodigoPostalRegionCelular` / `VTASCCRegionSku`).
 - **Pickup en Sucursal:** Está pendiente porque aún no se ha realizado la definición del flujo; esto se tiene que revisar y acordar directamente con el señor Sergio.

7. Refactorización Arquitectónica Async/Await (02/07 - 03/07):

- **Problema Detectado:** El sistema utilizaba llamadas síncronas (`.Result` o `.Wait()`) a `HttpClient`, lo que podía provocar *socket exhaustion* (fugas de puertos) y cuellos de botella en el servidor ASP.NET bajo alta concurrencia.
- **Decisión:** Refactorizar el 100% de las capas `Methods` y `Controllers` a operaciones `async Task`, implementando `using` en los objetos HTTP y `ConfigureAwait(false)`. Se dividió en 5 fases.
- **Progreso Realizado:**
 - **Fase 1 y 2 (Completado):** `Values`, `Account`, `Etiquetas`, `WalletCustomer`.

- **Fase 3 Parcial (Completado):** `BusinessPartner`, `PartnerAddress` (Direcciones de entrega) y `Sale` (Ventas). Quedan pendientes únicamente `Login` e `Imagen`.
- **Fijación de Bugs Críticos:** Se corrigió el "bug de la diagonal" que concatenaba mal la Base URL con el servicio OData, devolviendo error 404 en SAP Gateway.
- **Bloqueos de Compilación (`OrderMethods.cs`):** Para no romper la compilación general mientras se avanza por fases, se aplicó temporalmente `.GetAwaiter().GetResult()` en `OrderMethods` a los métodos recién convertidos a `Async`.
- **Hallazgo Pruebas Unitarias (03/07):** La API de creación de BP en SAP respondió con el error *"Entrada no existe en TFACD"*, evidenciando una falta de configuración del Calendario de Fábrica en el entorno de desarrollo SAP. El código C# envió el payload intacto; el bloqueo es de Basis/Funcional SAP.
- **Hallazgo Pruebas Unitarias (06/07 - Parámetro Fantasma `Perrl`):** SAP añadió el campo `Perrl` como mandatorio en los modelos de cliente sin emitir ningún aviso previo. Esto provocaba errores al enviar la petición. Tras investigar de dónde provenía y cuál era el nuevo parámetro exigido por S/4HANA, se añadió la propiedad al modelo `Client` y se inyectó el valor fijo "AM" en su inicialización (`BusinessPartnerMethods.cs`) para destrabar el flujo.
- **Hallazgo Pruebas Unitarias (06/07 - Bug de Migración en `UpdatePhone`):** Durante la validación, se descubrió que el método `UpdateAddressPhoneNumberAsync` en C# apuntaba erróneamente a la ruta de creación de BP (`ZAPI_BP01_PARTNER_SRV`) en lugar de a la ruta oficial para actualizar teléfonos (`/API_BUSINESS_PARTNER/A_AddressPhoneNumber`), exigiendo todo el body `Client`. Se reescribió el método y el controlador para recibir `addressId`, `person` y `ordinalNumber` y realizar un método `PATCH` nativo, dejándolo idéntico a la implementación original en Python.

8. Consolidación de Documentación y Brechas Funcionales (07/07):

- **Auditoría de APIs en Código C#:** Se escanearon los controladores y métodos del proyecto confirmando que las APIs del catálogo (`DM01`, `DM02`, `DM04`, `DM05`), precios (`SD29`), ATP (`DIM11`), Bonificaciones (`SD33`) y monederos (`SD18`) están **100% implementadas**.
- **Documentación de Parcialidades (Splits `TZ01`):** Se incorporaron las especificaciones técnicas (`tz01_zsplits_credilana.md`, `mercaderias`, `seguros`) al repositorio `RSG/`, aclarando cómo SAP sustituirá el flujo de divisiones logísticas/financieras. Queda pendiente la programación en C#.
- **Gestión de Zonas Grises y Bloqueos:** Se redactó el documento formal `email_validaciones_sap.md` dirigido a los consultores de SAP. Éste segmenta las 25 tablas locales pendientes por rubro de negocio y solicita directamente las equivalencias técnicas (ej. para Mutación de SKUs de Telefonía) necesarias para finalizar la migración.

Cronograma General de Migración SAP (.NET 4.7.2)

Este documento centraliza el calendario oficial de actividades estructurado **por día hábil**. Documenta el esfuerzo invertido en el imprevisto de las APIs de entrega y proyecta las fases futuras con un mapeo exhaustivo basado en el análisis real del código fuente actual.

Análisis de Código Fuente (Realidad vs Planeación)

🔗 Important

Auditoría de Código (02/07/2026): Tras escanear el proyecto `.NET` (`ServicioSap`), se confirma que **NO EXISTE** código previo, controladores ni DTOs para las APIs `EX01`, `TZ01`, `SD46`, `SD48`, Facturación ni Contacto. Todos los módulos a partir del 07/07 iniciarán desde cero absoluto (0% de avance en código). Esto confirma que las estimaciones deben contemplar desde la creación de las carpetas/modelos hasta las pruebas E2E.

Fase 1: Desvío Operativo - APIs de Entrega (Planeación)

Duración: 6 Días Hábiles | **Fechas:** 29/06 al 06/07

Contexto: Todo el esfuerzo se pausará en los demás módulos para priorizar el diseño e integración de las nuevas APIs de direcciones y la migración de cupones.

Día	Fecha	Actividad a Realizar	Responsable
Día 1	Lu 29/06	Se mejorará el proceso de consulta de direcciones de cliente para la obtención correcta del AddressId . Se integrarán las 6 nuevas APIs y se creará el PartnerAddressController .	Marcos Galindo
Día 2	Ma 30/06	Se realizarán las pruebas directas a SAP de las primeras 3 APIs y se integrará la API DM07 para la consulta de sucursales.	Marcos Galindo
Día 3	Mi 01/07	Se crearán los modelos (CouponModels) y se mejorarán el OrderController y sus métodos para adaptar el manejo de los cupones de ventas y sus reglas de negocio.	Marcos Galindo
Día 4	Ju 02/07	Se añadirán las credenciales a webconf para las nuevas APIs y se cambiará la lógica en los métodos de cupones para mejorar el flujo y la respuesta de la información.	Marcos Galindo
Día 5	Vi 03/07	Se culminará la implementación de cupones y se vinculará la dirección de entrega final al payload maestro del servicio SD01 .	Marcos Galindo
Día 6	Lu 06/07	Se ejecutarán End-to-End validando que las APIs de direcciones y cupones fluyan exitosamente hasta SAP.	Marcos Galindo

(Nota: Mutación de SKUs Telefonía y Pickup en Sucursal quedaron en "Hold" permanente con Dirección).

Fase 2: Módulo "Pago en Línea" (Abonos)

Duración: 14 Días Hábiles | **Fechas Reales:** 07/07 al 24/07

Responsables: Marcos Galindo, Diego Valle

Contexto: Al no existir código previo, el equipo deberá configurar los esquemas asíncronos (async/await) desde cero para OData V4 (TZ01) y REST (EX01).

Estatus de Documentación:  **Completada** (Ver RSG/ex01_documentos_no_compensados.md y RSG/tz01_zsplits_mercaderias.md).

Día	Fecha	Actividad Detallada (Desarrollo desde cero)	
Día 1	Ma 07/07	Arquitectura Base Abonos: Creación de Controladores, carpetas de Modelos y configuración genérica de HttpClient con OAuth para CPI.	
Día 2	Mi 08/07	API EX01 (Parte 1): Mapeo de DTOs para DocNoCompSet (Cuentas por cobrar).	
Día 3	Ju 09/07	API EX01 (Parte 2): Implementación del método GET con inyección de parámetros \$filter (KUNNR , Contable).	
Día 4	Vi 10/07	API EX01 (Parte 3): Pruebas de consumo de documentos no compensados.	
Día 5	Lu 13/07	API TZ01 Mercaderías (Parte 1): Mapeo DTOs de tabla ZME_SPLITS y endpoint \$batch .	
Día 6	Ma 14/07	API TZ01 Mercaderías (Parte 2): Implementación de POST y PATCH (Afectación de abonos).	
Día 7	Mi 15/07	API TZ01 Mercaderías (Parte 3): Pruebas unitarias de Mercaderías.	
Día 8	Ju 16/07	API TZ01 Préstamos: Mapeo de tabla Z y métodos CRUD análogos.	
Día 9	Vi 17/07	API TZ01 Seguros: Mapeo de tabla Z y métodos CRUD análogos.	
Día 10	Lu 20/07	Lógica de Negocio Abonos: Interconectar respuesta EX01 con consumo TZ01. (Inicia traslape con otros módulos).	
Día 11	Ma 21/07	API STP Clabe (Parte 1): Mapeo e integración de API Estándar STP.	
Día 12	Mi 22/07	API STP Clabe (Parte 2): Finalización lógica STP.	
Día 13	Ju 23/07	Pruebas Integrales (Parte 1): E2E Pago en Línea.	
Día 14	Vi 24/07	Pruebas Integrales (Parte 2): Liberación de Módulo de Abonos.	

Fase 3: Módulos Concurrentes (Traslape Final de Julio)

A partir del **Lunes 20 de Julio**, el equipo se dividirá para abordar el resto de APIs que tampoco cuentan con código base.

Cancelaciones y Devoluciones (APIs SD46, SD48)

Estatus:  **Completadas y Listas** (Desarrollo finalizado anticipadamente por Javier Alvarado). Listas para pruebas E2E.

Facturación

Duración: 5 Días Hábiles (20/07 al 24/07) | **Responsables:** Marcos G., Javier A.

Fechas	Actividad
20/07 al 21/07	Arquitectura de timbrado/generación, mapeo DTOs.
22/07 al 24/07	Pruebas de integración de facturas emitidas.

Formulario de Contacto

Duración: 5 Días Hábiles (20/07 al 24/07) | **Responsables:** Marcos G.

Fechas	Actividad
20/07 al 24/07	Endpoint sencillo de captura de leads/contacto Magento a SAP (Si aplica CRM).

Gestión de Información SEO y Etiquetas

Duración: 5 Días Hábiles (27/07 al 31/07) | **Responsables:** Marcos G.

Fechas	Actividad
27/07 al 31/07	Exposición de metadatos de productos / categorías para frontend Magento.

Fase 4: Zonas Grises y Cierre de Flujo (APIs Pendientes)

Duración Estimada: Por definir | **Responsable:** Marcos Galindo, Sergio, Luis Ricardo.

Contexto: Para poder dar por cerrado y 100% operativo el flujo de creación de órdenes (SD01), es obligatorio integrar la lógica de

Recoger en Sucursal (Store Pickup) y la mutación de SKUs (Telefonía), mismas que quedaron pausadas en la Fase 1.

Hito	Actividad Pendiente (Por agendar)	Dependencia
Definición de Tablas/APIs	Junta de resolución para acordar las tablas destino o los endpoints de S4HANA que recibirán los datos de Pickup y las validaciones de regiones para telefonía.	Sr. Sergio / Luis Ricardo
Desarrollo Store Pickup	Mapeo de DTOs e inyección de datos de sucursal de recogida dentro del método BuildSapOrder() y/o invocación a nuevas APIs de Pickup en C#.	Definición de Negocio
Integración SKUs Telefonía	Rehabilitación de la lógica de reemplazo de regiones (R5/R6) consultando el nuevo catálogo en SAP.	Catálogo S4HANA
Pruebas Finales SD01	Test E2E incluyendo compras de telefonía y entregas en tienda (Pickup) comprobando que viajan correctamente hacia SAP.	Fase 4 Completada

Nota Técnica General: Acatando la directriz oficial de arquitectura, todos los módulos de las Fases 2, 3 y 4 serán estructurados obligatoriamente con patrones asíncronos (async Task , await HttpClient.SendAsync) para garantizar el máximo performance y anular el riesgo de deadlocks.

Cronograma de Migración a SAP

Proyecto: Migración de Lógica LAN/Intelisis → C# (Servicio nuevo) para Creación de Órdenes (Contado y Crédito MA)

Fecha Inicio: 27/05/2026

Fecha Fin: 12/06/2026 (con posible extensión por integraciones entre servidores y servicios externos)

Descripción

Se realizará la migración y adaptación del flujo SetPedido (LAN) hacia el servicio nuevo en C# integrado con SAP, contemplando dependencias actuales en SQLite (OpenPay/guías), SQL Server Intelisis (SPs y datos operativos), SQL Server Android/MAVIANDROID01 (crédito web + SMS) y servicios externos (Liberador crédito y DMZ Magento callback). El objetivo de este bloque es dejar operativa la base del flujo (contado) y avanzar en el flujo de crédito MA validando qué SP/tablas siguen vigentes o deben reemplazarse por APIs SAP (por ejemplo, la equivalencia de obtenerIdVenta).

Objetos involucrados (SPs / Tablas / Servicios)

SQLite

Tablas: openpay_orders, openpay_stores, servicio_guías

SQL Server Intelisis

Tablas: DM0312DatosEntrega, CteTel, TablaStd, TrWDM0285_CteRecoge, VTASdArtCreditoWeb

SP/XP: SpVTASeCommerceDetPedidos, SP_eCommerceNuevoPed, SPVTASPedidosMagento, SpWDM0285_CteRecoge, SpVTASVentaCupon, SpVTASInsertArtSolCreditoLinea, xpVerificarMovMonederoMAVI, SP_DM0312TarjetaSerieMovMAVI, spGenerarMovMonederoMAVI, spAfectar

SQL Server Android / MAVIANDROID01

Tablas: TcAAEA00030_EnvioMensajes, VTASDCodigoVerificacionCommerce

SP: SP_CREDITO_WEB_DATOS, SpCREDIDatosSolicitudCreditoArt

Servicios externos

Liberador crédito: AUTENTICACION_URL_LIBERADOR, VETA_URL_LIBERADOR

DMZ Magento: login/authenticate, order/authorizationResult

SAP: API/servicio SD01 para equivalencias de consulta/validación

Plan de desarrollo (27/05/2026 al 12/06/2026) — sin fines de semana

Miércoles 27/05/2026

- Se realizará el desglose del flujo LAN SetPedido por módulos migrables: normalización SKU, validaciones iniciales, OpenPay, forzarOrder, envío/pickup, detalle de partidas, guía, crédito (SMS + solicitud + artículos), liberador/callback, creación final y post-proceso.
- Se definirá el orden de implementación por criticidad (contado primero y crédito como bloque validable por dependencias).

Jueves 28/05/2026

- Se migrará la lógica de AgruparCantidadPorSKU al servicio nuevo (sumatoria de cantidades por SKU y acumulado de descuento).
- Se migrará el armado equivalente a ToArray a una estructura tipada, manteniendo la regla de negocio: precioEspecial != 0 fuerza forzarOrder = 1.

Viernes 29/05/2026

- Se dejará en validación la equivalencia de obtenerIdVenta debido a que la tabla Venta ya no existe en el flujo objetivo.
- Se implementará y probará la equivalencia usando API de SAP SD01 para:
 - validar si un pedido ya existe (idempotencia por incrementId/referencia),
 - definir el campo SAP que funcionará como llave de consulta.
- Se ajustará el retorno "PedidoExistente" para depender de la consulta a SAP y no de Intelisis.

Lunes 01/06/2026

- Se migrará la lógica de OpenPay incorporando:
 - configuración de conexión a SQLite en el servicio nuevo (ruta/servidor, credenciales si aplican),

- validación de conectividad al servidor del .db,
- pruebas de inserción en openpay_orders y openpay_stores.
- Se integrará la detección de envío vs pickup (instore_pickup) y se dejará en análisis la dependencia del SP SpWDM0285_CteRecoge para determinar su función exacta y cómo se representará en el flujo nuevo.

Martes 02/06/2026

- Se migrará el procesamiento de partidas:
 - validación precio > 0 y cantidad > 0,
 - acumulado fTotalDescuento,
 - mapeo de partidas a estructura SAP (items/condiciones/textos).
- Se analizará el SP SpVTASCommerceDetPedidos (opciones Limpiar/Insertar, respuestas Result/PrecioIncorrecto) para identificar reglas internas que deben conservarse o replicarse en C#.

Miércoles 03/06/2026

- Se migrará el registro de guía (SaveGuide) y se validará la inserción en servicio_guías (SQLite) con INSERT OR IGNORE.
- Se migrará la preparación del flujo crédito para artículos:
 - armado de lista cantidad,sku,
 - inserción del concepto SEGU00001 cuando costoEnvío != 0.

Viernes 05/06/2026

- Se validará si las tablas y consultas usadas en la validación SMS siguen aplicando en la migración:
 - Android: TcAAEA00030_EnvioMensajes y VTASDCodigoVerificacionCommerce,
 - Intelisis: CteTel y la regla IsInTableStd (TablaStd + CteTel).
- Con base en esa validación, se implementará o ajustará la lógica de:
 - obtención de número asociado al SMS,
 - obtención de teléfono móvil validado del cliente,
 - regla isValidated y el cálculo de ValidacionTelefono.

Lunes 08/06/2026

- Se validará si los SPs actuales de crédito y sus tablas se continuarán utilizando en el flujo nuevo o si se reemplazarán por SAP:
 - SpCREDIDatosSolicitudCreditoArt (CheckCliente/GetSaldo/GetInfo),
 - SP_CREDITO_WEB_DATOS (Android/MAVIANDROID01).
- En caso de mantenerse, se integrará la ejecución controlada desde el servicio nuevo (conexiones, timeouts, manejo de errores y retorno consistente del proceso).

Martes 09/06/2026

- Se integrará el registro de artículos en solicitud de crédito (si aplica mantenerlo):
 - ejecución de SpVTASInsertArtSolCreditoLinea por artículo,
 - regla especial para SEGU00001 (cantidad=1, SeguCost=costoEnvío),
 - regla anti-duplicado por artículo.
- Se integrará la operación de promotor/cupón en crédito:
 - ejecución de SpVTASVentaCupon con opcion="Elimina" cuando exista codigo_promotor.
- Se evaluará el ajuste requerido de la regla cliente.StartsWith("C") para transición a BP/Partner SAP.

Miércoles 10/06/2026

- Se integrará el servicio de Liberador de crédito:
 - autenticación,
 - envío a venta y manejo de respuesta (EN_ANALISIS/RECHAZADO, idVenta).
- Se integrará la notificación a Magento por DMZ:
 - login/authenticate,
 - order/authorizationResult con reintentos y trazabilidad por entityId.

Jueves 11/06/2026 (posible extensión)

- Se realizarán pruebas integrales del flujo crédito entre servidores (servicio nuevo ↔ BD Intelisis ↔ BD Android ↔ Liberador ↔ DMZ).
- Se ajustarán configuraciones de red, credenciales, timeouts, y manejo de errores por disponibilidad/latencia de servicios externos.

Viernes 12/06/2026 (posible extensión)

- Se realizarán pruebas de estabilización del flujo completo (contado + crédito):
 - idempotencia por SD01,
 - OpenPay + SQLite,
 - crédito + SPs/servicios externos (si siguen vigentes).
- Se evaluará el bloque de monedero/afectación (OpenPay/PayPal) para definir su integración inmediata o su calendarización, debido a dependencias en SPs (xpVerificarMovMonederoMAVI, SP_DM0312TarjetaSerieMovMAVI, spGenerarMovMonederoMAVI, spAfectar) y necesidad de pruebas completas de ambiente.

10. Corrección de Flujo Contado y Monedero (Sesión Actual)

- **GAP 1: Bug en OpenPay Stores (Paynet):** Se eliminó el *Early Exit* (return prematuro) en OrderMethods.cs para el método de pago OPENPAY_STORES_METHOD. Ahora el código emula correctamente a LAN: guarda el registro en SQLite pero continúa la ejecución para que la orden sí se genere en SAP S/4HANA.
- **GAP 2: Monedero Electrónico (Generación Rota):** En el código migrado, la activación de monedero para órdenes exitosas de PayPal/OpenPay estaba validando erróneamente un `ildVenta > 0` que siempre era cero. Se refactorizó la lógica invocando `GenerarMonederoSAP(incrementId, ...)`. El método internamente está como cascarón (sin lógica activa) dejándolo pendiente hasta que se defina la API OData exacta que SAP proporcionará para afectar el Monedero.
- **Estado de Pickup:** Queda registrado que la vinculación del pedido a la entrega (`ddrchange`) ya opera en SAP, pero la recopilación de datos de la persona que recoge (RegisterPickupClientInfo, homólogo a `setNameToReference` de LAN) queda pendiente de integración o definición de la API de Destino.
- **Estado de Crédito (Liberador):** Se confirma que el método `ProcessCreditPayment` (que escribe a la BD de Android) es correcto y se mantiene tal cual. La parte que notifica a Magento y lanza el `liberadorThread` está pendiente a la espera del desarrollo del componente 'Liberador' por parte de otra área.

11. Estructuración del Módulo de Abonos (Fase 2 - Consultas EX01 y TZ01) #migracion #SAP #dotnet

- **Modelos y Controladores:** Se crearon `AbonosController.cs` con el prefijo `credit/` para asegurar retrocompatibilidad con las peticiones frontend de Magento (`getClientSaldo/{cliente}` y `getClientFactura/{cliente}/{factura}`).
- **Mapado de Datos:** Se implementaron los DTOs `DocNoCompResponse.cs` para el payload de la EX01 y `ZSplitDto.cs` para la TZ01, reflejando exactamente las estructuras devueltas por S/4HANA OData.
- **Reglas Arquitectónicas (SKILL):** Se reestructuró `AbonoMethods.cs` eliminando cualquier rastro de URLs hardcodeadas y variables de entorno quemadas. Ahora se utiliza estrictamente `Conexion.Data.obtenerUrl` y `TokenGenerator.CreateClientS4()` para obtener la URL base dinámicamente y el `HttpClient` securizado contra SAP Gateway, respetando la *Regla de la Diagonal* para evitar errores 404 OData y la *Regla de Abstracción*.
- **Limpieza de Errores de Compilación:** Se deshabilitó / comentó temporalmente la lógica zombie de RMAs/Cancelaciones (`ReverseGoodsIssue` y `CancelInvoice`) en `OrderController.cs` debido a que generaba errores de compilación por falta de implementación en `OrderMethods.cs`, dejándola aislada para su futura fase.
- **Siguiente Acción Pendiente:** El usuario realizará peticiones en Hoppscotch para inyectar IDs reales de S4HANA y recopilar los payloads JSON que certifiquen el éxito de la conexión en `master_test_plan.md`.

12. Creación Automática de BP, Centros Dinámicos y Logs Seguros (Sesión Actual)

- **Alineación de Creación de Business Partner (BP):** Se sincronizó el payload de las Órdenes de Magento con el endpoint de BP. Se igualó el método `BuildBpClientFromOrder` inyectando campos obligatorios (`Perrl` , `Marst` , `Gender`) y estructuras vacías (`toCte` , `toCteCto` , `toCteTel` , `toCteCtoEmpleo`). SAP ahora crea el BP automáticamente y lo asigna al `Zctefinal` de la orden de forma nativa sin tronar la petición.
- **Asignación Dinámica de Centros (Plant / Werks):** Se inyectó la matriz de decisión definitiva en la validación de stock (`DeterminarPlant`) y armado JSON (`BuildSapOrder`). Mapeo implementado: MA Contado (`0090`), MA Crédito (`0504`), VIU Contado (`0041`), VIU Crédito (`0505`).

- **Arquitectura de Logs Segura (Local & Stage):** Se refactorizaron `Logger.cs` y `GeneradorLog.cs` con bloques `switch`. Ahora permite aislar los logs locales dentro del `BaseDirectory` del proyecto previniendo caídas del sistema por falta de permisos o rutas inexistentes (vs `C:\inetpub`).

Informe de Estatus y Solicitud de Apoyo (Migración LAN a SAP)

Fecha de corte: Julio 2026

Objetivo del Documento: Solicitar el apoyo de los compañeros de equipo para cubrir los puntos faltantes del cronograma y destrabar dependencias clave, garantizando la entrega final del 31 de julio.

1. Resumen de Avances (Lo que ya está listo)

Se ha completado la migración base de la lógica de órdenes (de Base de Datos Intelisis a Código C#) de acuerdo al cronograma inicial (20/05 - 12/06) y las reglas de cuentas C de Magento (15/06 - 26/06):

- **Delegación Financiera Exitosa:** Se erradicó la dependencia del `spAfectar`.
- **Módulo de Crédito y SMS:** Lógica validada cruzando `MAVIANDROID` y `BP05`.
- **Persistencia de Direcciones OData:** Se eliminaron las tablas heredadas (`DM0312DatosEntrega`).
- **Adaptación a Tecnología Async (Fases 1 a 3):** Se adaptó gran parte del proyecto hacia el modelo Asíncrono (`async/await`) para soportar alta concurrencia.

2. Puntos Críticos y Apoyo Requerido

El proyecto tiene las siguientes dependencias o bloqueos en donde requiero el apoyo de los compañeros para poder avanzar:

✓ A. Error de SAP Basis en Calendario de Fábrica (TFACD) - [RESUELTO POR DESARROLLO]

- **Problema Original:** La ruta para crear clientes devolvía el error *"Entrada no existe en TFACD"*.
- **Resolución:** A pesar de ser un error de SAP, el equipo de desarrollo investigó y descubrió que SAP S/4HANA exigía un parámetro oculto no documentado (`Perrl`). Al inyectar el valor fijo `"AM"` desde C#, se logró saltar la validación del calendario de fábrica y **la creación de clientes ya es exitosa**. Ya no se requiere apoyo de Basis para este punto.

● B. Falta de Definición de Tablas (Telefonía y Pickup)

- **Problema:** No se me ha definido el listado final de tablas que voy a utilizar y cuáles se descartan para la mutación de SKUs (celulares) y la recolección en tienda (Pickup).
- **Contexto:** Actualmente cuento con un Excel con el listado de tablas de Intelisis, pero sigo a la espera de que se me indique oficialmente qué tablas o APIs sustituirán estas operativas.
- **Apoyo Solicitado:** Dado este análisis de lo que hace falta implementar comparado al proyecto actual, solicito justificadamente que los compañeros entren al proyecto para ayudarnos a cubrir estas funcionalidades.

● C. Desfase de Datos Maestros (Cupones)

- **Problema:** La parte del código de validación de cupones está correcta y funcional, pero los catálogos en AWS contienen claves obsoletas de Intelisis (ej. `D40`), mientras que las APIs de Android exigen identificadores actualizados de SAP SuccessFactors (ej. `20000001`).
- **Apoyo Solicitado:** Apoyo para alinear y actualizar los catálogos de "Códigos de Promotor" en AWS.

● D. Adaptación a Tecnología Async (Trabajo Restante)

- **Aclaración:** No se van a refactorizar los modelos ni los métodos actuales. El trabajo consiste **únicamente** en adaptarlos a la tecnología `async`.
 - **Impacto:** Este trabajo requiere tiempo que compite con las fechas apretadas de los demás entregables (Facturación, Abonos, etc.).
-

3. Cronograma de Entregas Restantes (Rumbo al 31 de Julio)

Para dimensionar la necesidad de apoyo, este es el cronograma de los módulos en los que debo concentrarme este mes de julio:

Módulo / Tarea	Fecha de Inicio	Fecha de Fin	Estatus
Migración de Lógica (BD Intelisis a C#)	20/05/26	12/06/26	⚠️ Atrasado (Impactado por integración de 7 APIs nuevas: Direcciones y DM07)
Reglas Cuentas C de Magento	15/06/26	26/06/26	⚠️ Atrasado (Pruebas del flujo completo SAP-DMZ aún en curso)
Pago en Línea (Abonos) (APIs TZ01_ZMER, TZ01_ZPRE, TZ01_ZSEG, EX01, Cuenta clabeSTP)	29/06/26	24/07/26	✅ APIs conectadas a DMZ / ⚠️ CLABE bloqueada por definición (Álvaro)
Facturación	20/07/26	24/07/26	🕒 Pendiente
Formulario de Contacto	20/07/26	24/07/26	🕒 Pendiente
Cancelaciones y Devoluciones (APIS SD46, SD48)	20/07/26	31/07/26	✅ Completadas (Javier A.)
Gestión de Información SEO y Etiquetas	27/07/26	31/07/26	🕒 Pendiente

Conclusión:

Necesito encarecidamente que mis compañeros me apoyen resolviendo el tema de SAP Basis, la definición del Excel de tablas para Pickup/Celulares, y la alineación de AWS para Cupones. Solo destrabando esto podré enfocar mis tiempos en adaptar el sistema a asíncrono y en construir la enorme carga de los 5 módulos restantes del cronograma antes del 31 de julio.